BLA-06 · GENAI ARCHITECTURE

Putting It Together: GenAI Architecture Trade-offs

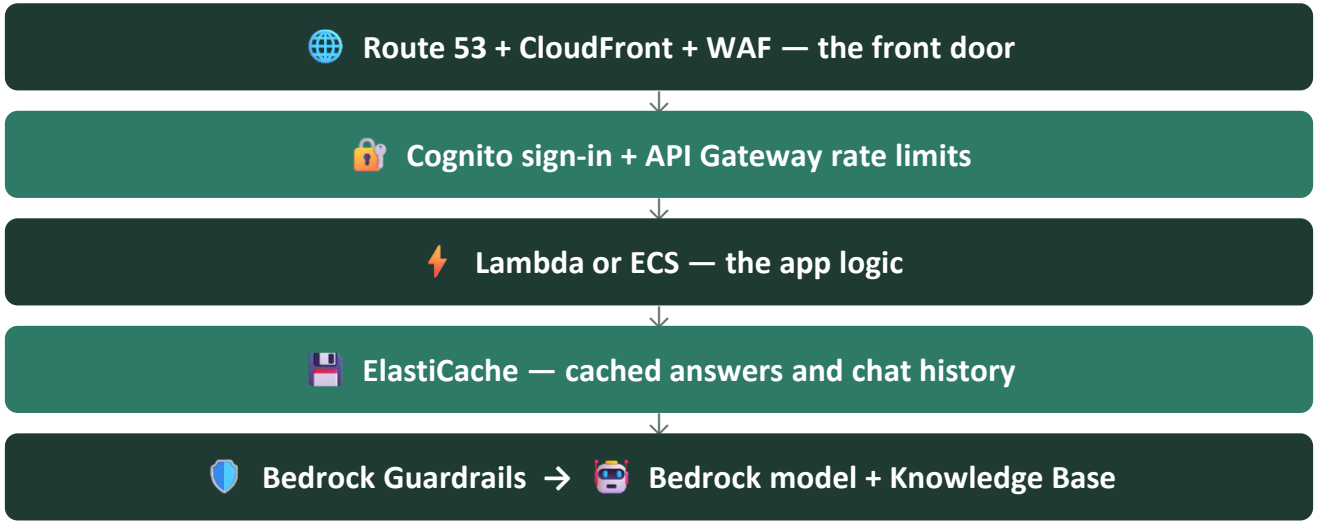
A complete user-facing system · Choosing a vector store · When to use Step Functions · Internal vs user-facing designs



PART 03

Smita Krishnan · AWS Generative AI Developer · Concordia University Chicago

A complete user-facing GenAI system



CloudWatch · X-Ray · CloudTrail watch the whole path.

Why each layer is there

Layer	Protects against	Without it
WAF + rate limits	Floods and abuse	One script runs up your token bill
Cognito	Unknown users	Anyone can use your model
Cache	Paying twice for one answer	Higher cost, slower replies
Guardrails	Harmful or off-topic content	Safety and brand risk
Observability	Not knowing what's wrong	Slow, blind debugging

💡 **Every layer adds cost, delay and upkeep.** Add it because of a real risk you can name.

Choosing a vector store

Somewhere to keep the embeddings for RAG.

Option	Strong at
OpenSearch Serverless	Usual default for Bedrock knowledge bases; keyword + meaning (hybrid) search
Aurora PostgreSQL (pgvector)	SQL and vectors together, when data already lives in Postgres
MemoryDB	Lowest latency, for real-time apps
Neptune Analytics	Answers that depend on relationships (GraphRAG)
Amazon S3 Vectors	Very large collections at low cost, when some delay is fine
DocumentDB	Apps whose data is already JSON documents

Questions that pick the vector store

Ask…	Leans towards
"Do we already run Postgres, or store JSON documents?"	Aurora pgvector · DocumentDB
"Do users search by exact words too, like product codes?"	OpenSearch
"Is every millisecond important?"	MemoryDB
"Are the connections between things the point?"	Neptune Analytics
"Millions of vectors and a small budget?"	S3 Vectors

💡 **Start with what your team already runs.** A new database is a new thing to learn, secure and pay for.

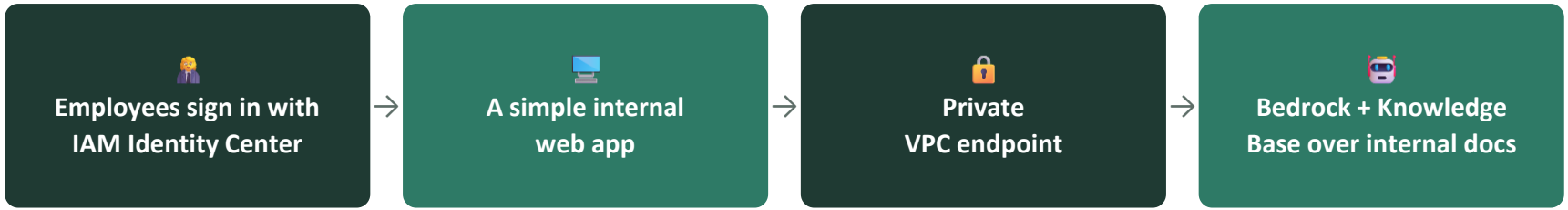
Orchestration: when to use Step Functions

Situation	Use
One model call, quick answer	A single Lambda function
Fixed steps in a known order	Step Functions
Waiting for a person to approve	Step Functions
The job runs longer than 15 minutes	Step Functions (Lambda's limit)
The model decides what to do next	Bedrock Agents or an agent framework

🏢 **Contract review:** Textract → summarise → Guardrails check → if risky, wait for a lawyer → save.

A simpler internal architecture

An assistant that helps staff search HR policies.



✂️ **Can leave out:** CloudFront, global routing, heavy rate limits

✅ **Must keep:** sign-in, document permissions, logging

Internal vs user-facing

	Internal tool	User-facing app
Users	Known employees	Anyone on the internet
Sign-in	IAM Identity Center	Cognito
Scale	Hundreds	Thousands to millions
Biggest risk	Data reaching the wrong colleague	Abuse, cost and harmful output
Design goal	Simple and cheap to run	Resilient, protected, fast worldwide

💡 **Same model, same knowledge base —** the difference is who's on the other end.

Key takeaways

1. Every layer should answer a real risk
2. No vector store is best — start with what you run
3. One quick call: Lambda. Fixed steps or waiting: Step Functions
4. Internal tools can be simpler — but keep permissions and logs
5. The users shape the architecture more than the model does

Start simple — add each layer for a reason